Advanced Topic 1: Git

We decided to explore Git as one of our advanced topics to achieve an easier workflow. Before starting the project, we had to accept that our code is very error prone since we have barely started coding in java; therefore, we concluded that each of us needed a secure environment in which to explore our ideas and try them out. Moreover, it was not feasible to work simultaneously on the same computer throughout the whole four weeks. Overall, it would not have been productive to not implement Git in our project.

We learned that Git is a system that manages the versions of a given project. Along with the website GitHub, it stores the files and changes that have been made on the cloud allowing revision if problems arise at some point in development. The software works though “branches” which are isolated workspaces that are built on top of previous code in another branch. This means that from a working basis two approaches can be tried out at the same time without disturbing the other, if one does not work out it can be disregarded for the time being. Since the project is stored in the cloud it makes collaborating easier for the users, each teammate can have their branch locally and “push” to the cloud when a goal is reached. Furthermore, branches can be “merged” to combine implementations form one to the other.

When it comes to the actual working with Git, we created a repository on GitHub. From the main branch we divided the work into two branches: front-end and back-end. We locally accessed the files; each teammate developed the agreed upon methods and classes in the corresponding workspace. If we felt like the change might disrupt everything that we had done so far then we simply created a new branch to explore the idea; if it worked out, we would implement it back to the branch.

After finishing the project using Git, we learned the importance of communication and organization. Firstly, since we did not work on everything simultaneously, we did not understand at first glance what the other had thought of, we needed clarity when naming variables and actual written explanations to guide the other. Secondly, working on the same branch was not an option since constant conflicts would arise, we had to decide what each branch’s purpose was, what could and could not be done and who would mainly use it. In conclusion, Git has improved our workflow, productivity, clarity and organization as well as giving us both our own breathing space to be creative without fearing messing up.

Advanced Topic 2: Multithreading

In the design of our project multithreading was necessary. Because our project is a music score editor and player, we could not leave out polyphonic melodies since they create the core of all music: harmonies. Without multithreading the playing of various notes would simply not be possible, and it would not be as responsive as it is in its current state.

Additionally, the design of the GUI includes multithreading implementation. During the execution of the note playing function, the accurate stop/pause function requires multithreading. Without multithreading, the user would have to wait until all the bars are done playing, since the pause action is initialized when play button is untoggled.

When researching we learned how programs are the sum of different parts that are being executed, these are called threads and the practice of using multiple of them is called multithreading. This idea helps improve performance since it makes the most of the CPU's capacity. However, since threads are executed at the same time, they have unpredictable behaviours in timing. For example, the first thread that has been created will not necessarily go first when performing a specific task. In fact, it is possible at some point in time that a thread's action is executed twice in a row.

Moreover, there are mainly two ways of building threads, ether by extending the Tread class or implementing the Runnable interface. When extending the Thread class, the class should override the run method, then the thread can be run. The other method, implementing Runnable, provides a wider range of possibilities by passing an instance of the Thread class to a constructor, and initializing a new object. Passing the Runnable object to the Thread and starting execution. We have implemented both ways of Thread construction: extension of the Thread class to initialize the Pause function and implementing the Runnable interface to play multiple notes at the same time.

From the implementation of Multithreading, we learned the importance of managing information inside our code. For example, a new class Phonetic had to be created on top of the organization that we had previously. Managing the threads was also a struggle, for example when stopping the playing of the sheet the notes would not turn off, instead they would keep going forever. More careful design decision ensued from these issues.